QualiAdept

Session 3: Vector Basis and Embeddings

Chapter 5: Artificial Intelligence, Vector Bases and Data Ingestion

> 📝 **In this chapter, we build the brain of our assistant.**
> So far we have successfully downloaded documents and kept them in the memory of the n8n platform. Now we want the Artificial Intelligence to read a large text document (a log file or documentation) and be able to answer questions extracted from it. We will learn how to use Vector Databases and "Chunking" techniques.

1. Problem: Why don't we send the entire document to ChatGPT?

As QA or DevOps engineers, we work with massive logs and miles of documentation. The first instinct is to send the entire document to a model like OpenAI (GPT-4) and tell it: *"Read it and answer my questions"*.

This approach is **wrong** for two major reasons:

- **Context Limit (Token Limit):** AI models have limited "short-term memory." They can't memorize an entire book in one go.
- **The huge costs:** OpenAI charges you for every word you submit. If you submit dozens of pages over and over, you'll quickly burn through your budget.

The Solution: RAG (Retrieval-Augmented Generation) Architecture

Instead of giving the AI the whole book, we will "cut" it into small paragraphs using a *Text Splitter* (Text Shredder). Then, we will convert these paragraphs into numbers (Vectors) and store them in a special database. When we have a question, we will extract *only* the relevant paragraph.

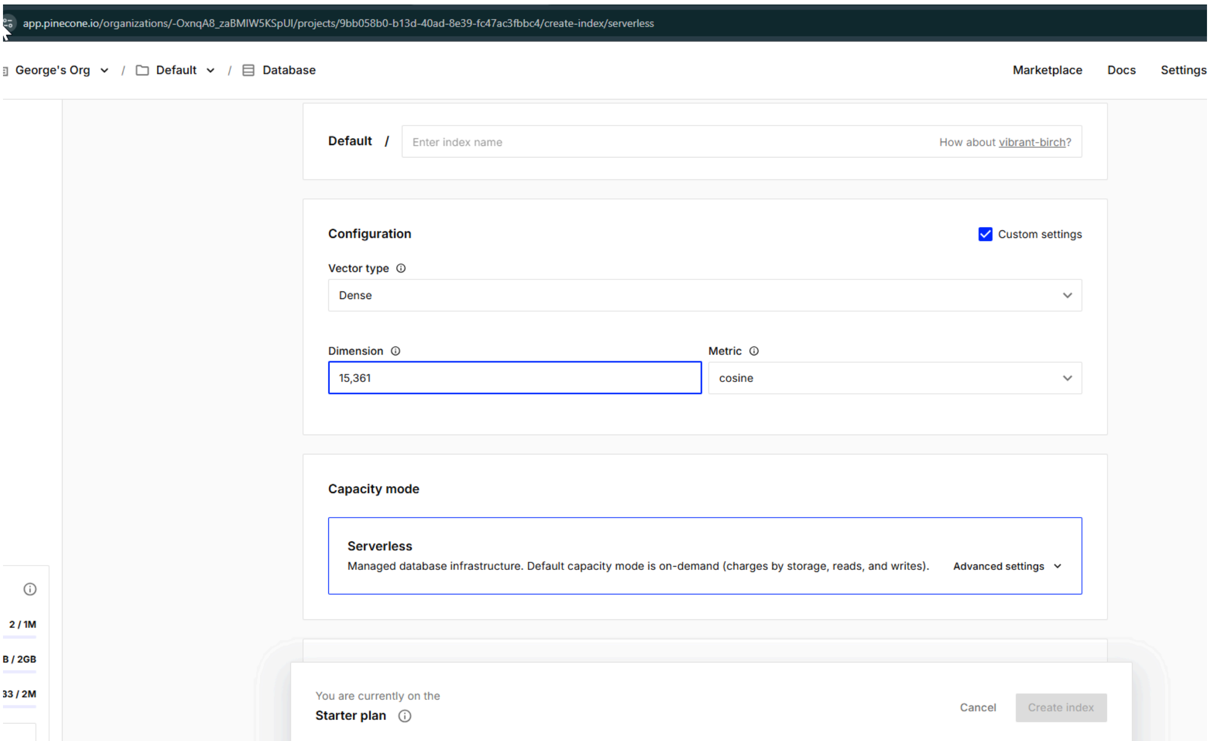
2. AI Theory: What are "Embeddings" and Vector Basis?

To a computer, words mean nothing. AI models understand the world through mathematics. The process of taking a word or sentence and translating it into a long string of mathematical coordinates is called **Embedding**.

Vector Database is the storage space optimized specifically for searching and comparing these strings of numbers at breakneck speeds. We will use **Pinecone** , the market leader in free vector bases.

3. Pinecone Setup: Creating the "Shelf" for Vectors

- Go on pinecone.io and create a free account.
- Click on **Create Index** (Creating your base).
- To the section **Name** , put the name: n8n-qa-docs.
- **Remark!** Check the box **Custom settings** (to the right of the section*Configuration*). This will unlock hidden options related to the size of the model.
- At the field **Dimension** newly appeared, write the value **1536** (The metric remains *cosine*). This is the fixed format required by OpenAI.
- The **Capacity mode**, make sure the free settings are active: **Serverless**, Cloud provider pe **AWS** and Region on **Virginia (us-east-1)**. Press **Create Index**.



4. Building the Ingestion Flow (Data Pipeline) in n8n

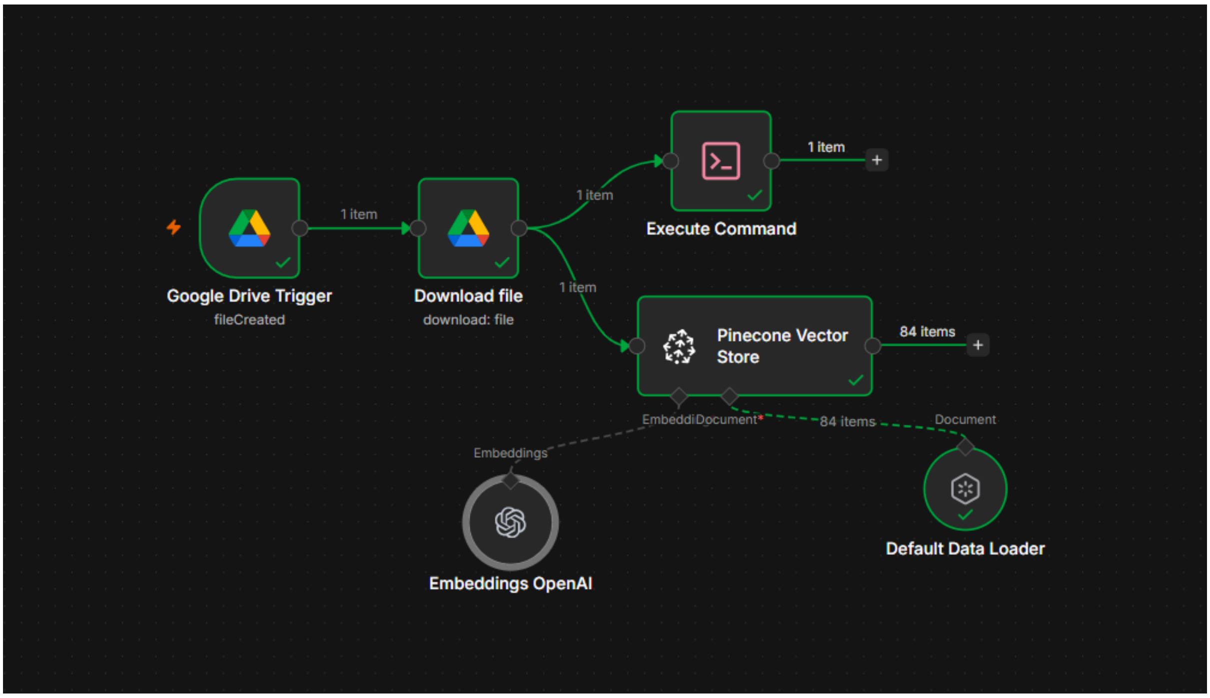
Now we go back to the n8n worksheet to connect all these pieces. Make sure to use the knots from the new category. **AI** (Artificial intelligence) nodes work like Lego pieces: we have a main node on the board, and below it we attach "sub-nodes" using the lower ports.

🚨 MAJOR ARCHITECTURAL TRAP:

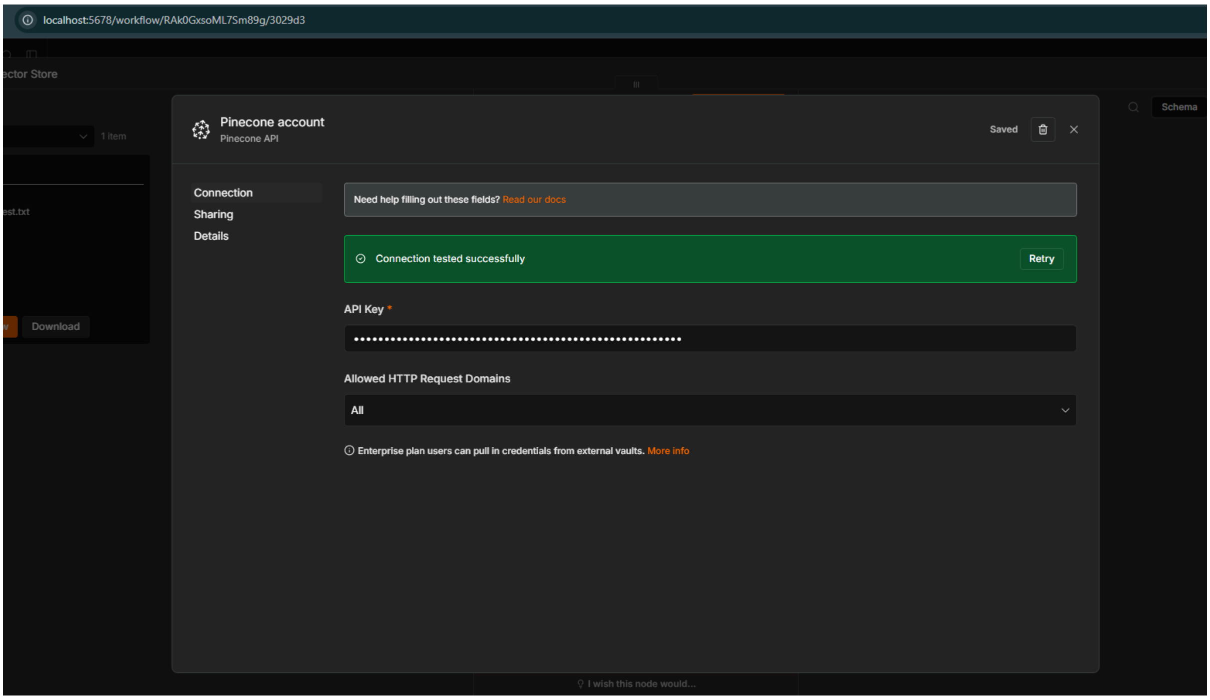
So far, we have tied the knots in a straight line. **DO NOT tie the Pinecone node after the Execute Command node!** The terminal node used the filename, but it "consumes" the raw text, sending an empty execution forward. The vector base needs the original file, so we need to create a **Branching (Bifurcation)**!

- **Main Node (Creating the Bifurcation):** Go to the node **Download file** (which contains the raw data). Grab the connector to its right and pull a **for now** next to the existing wire, so that two parallel paths are created. When you release the click, look for the category **AI** and choose **Pinecone Vector Store**.

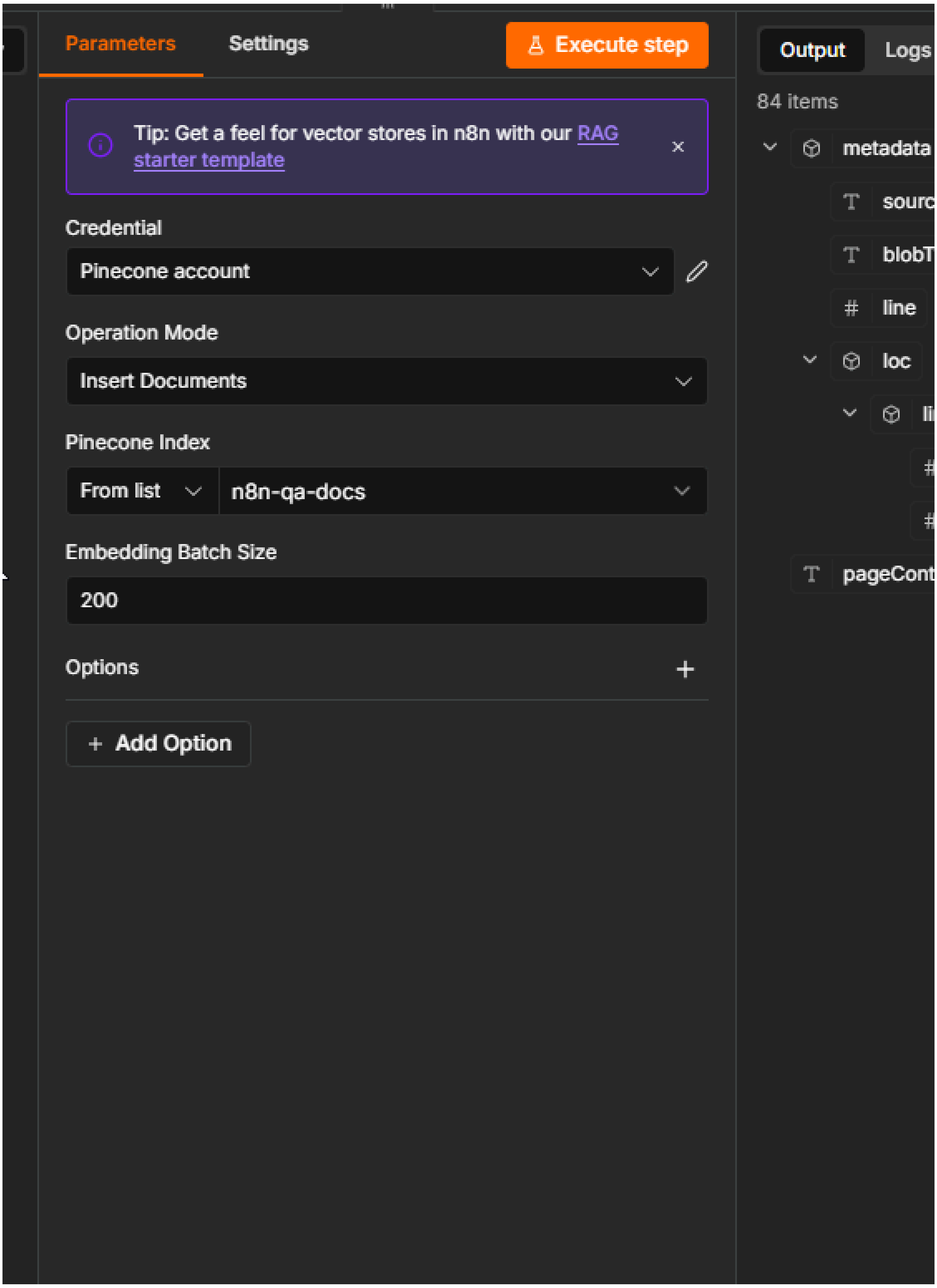
QualiAdept



- The credentials window will ask you directly for the **API Key**. Go to your Pinecone account (left menu -> API Keys), copy the long key and paste it into n8n. Save. The platform connects invisibly to the server.



- On the main operation, make sure it is selected **Insert Documents**.
- At the field **Pinecone Index**, click and choose the name of your newly created database from the list: n8n-qa-docs.

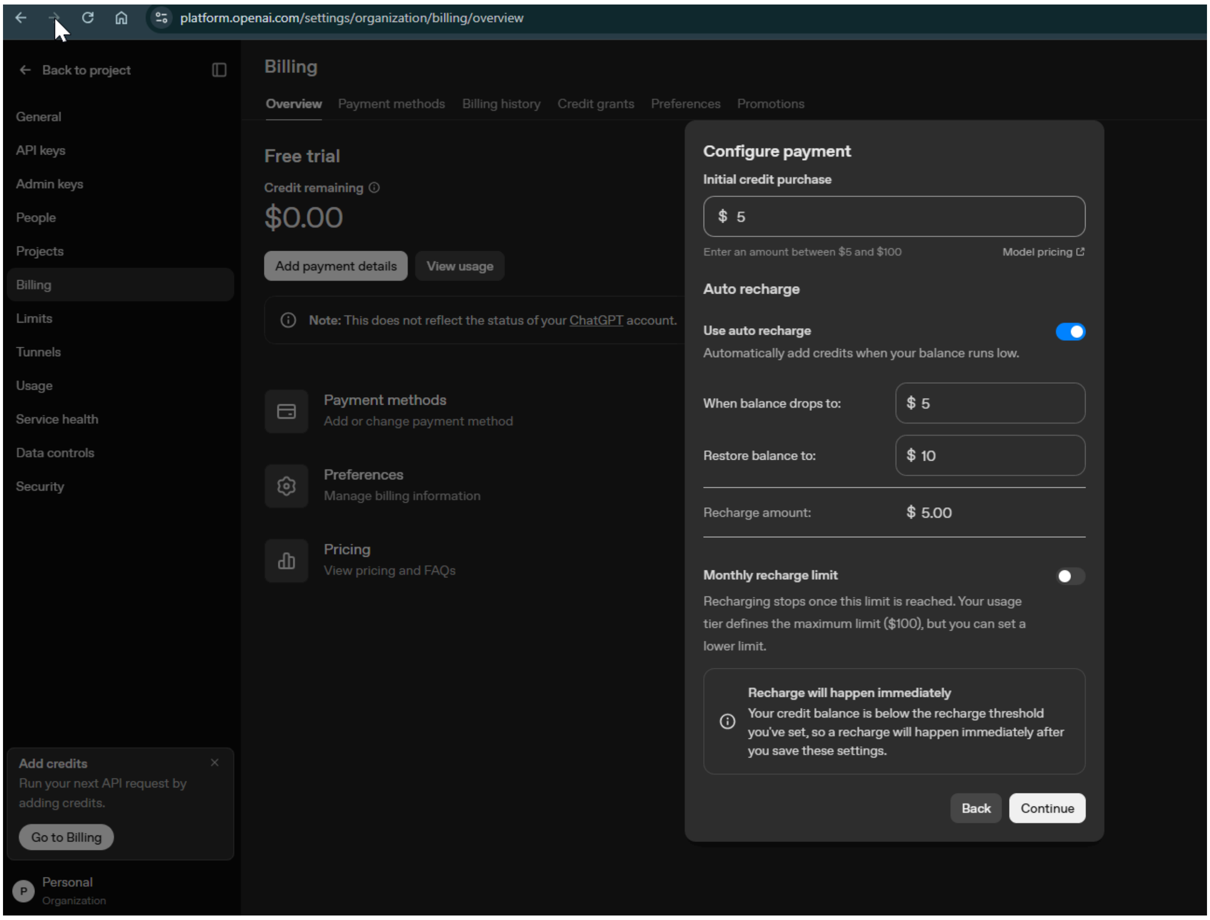


💡 Architect's Eye: The Two "Arms" of the Pipeline

You might ask: *"If we bypassed it for AI, why do we still keep the Execute Command node on the screen? "*The answer is that we didn't bypass it to cancel it, but we created a **parallel processing (Multitasking)**.

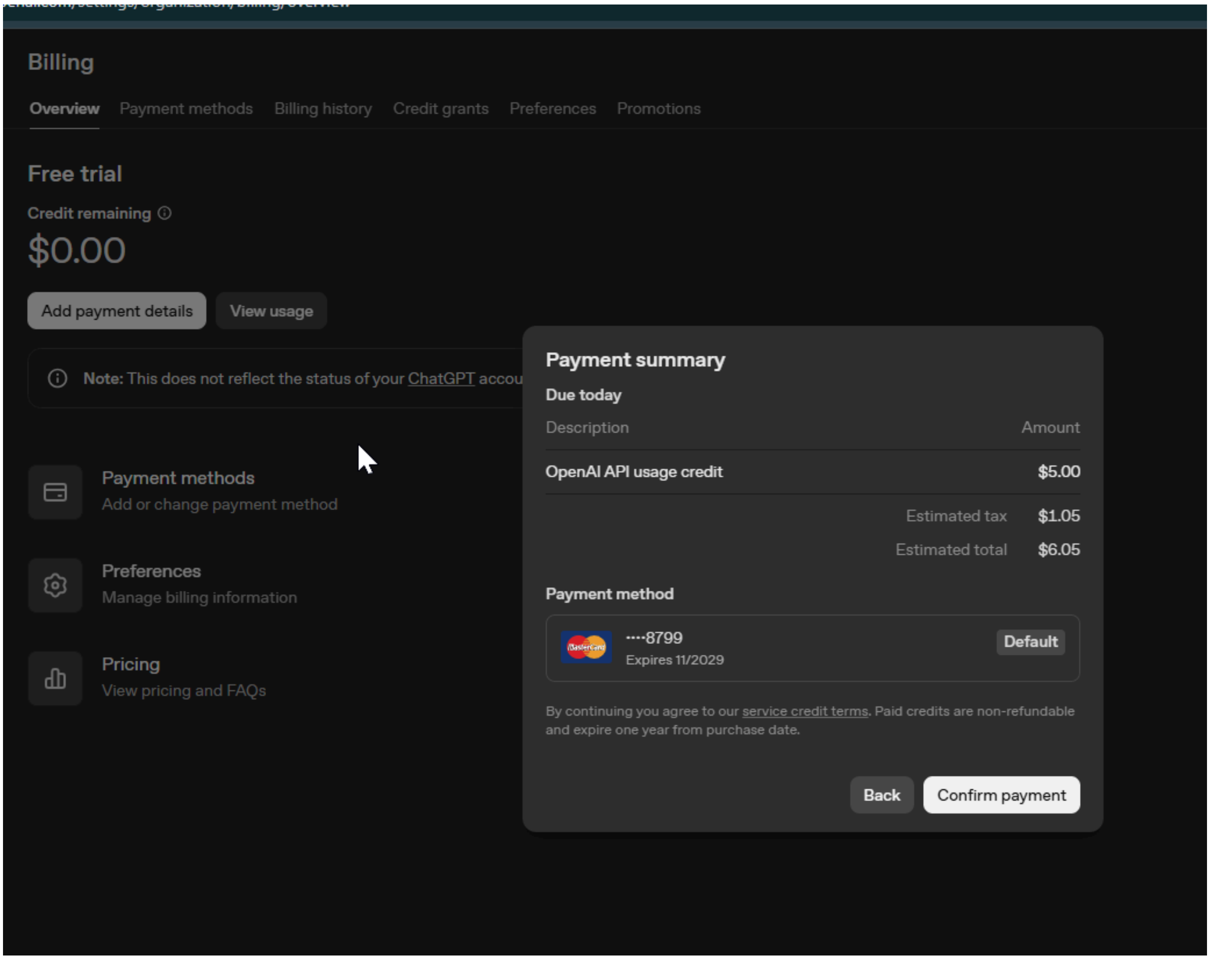
    - **Top Arm (DevOps Role):** Run the terminal command. Write it down locally on your hard drive.*the fact*that the file has arrived, for audit and traceability.
    - **Lower Arm (AI Role):** It takes the raw, untouched content and sends it for vectorization.

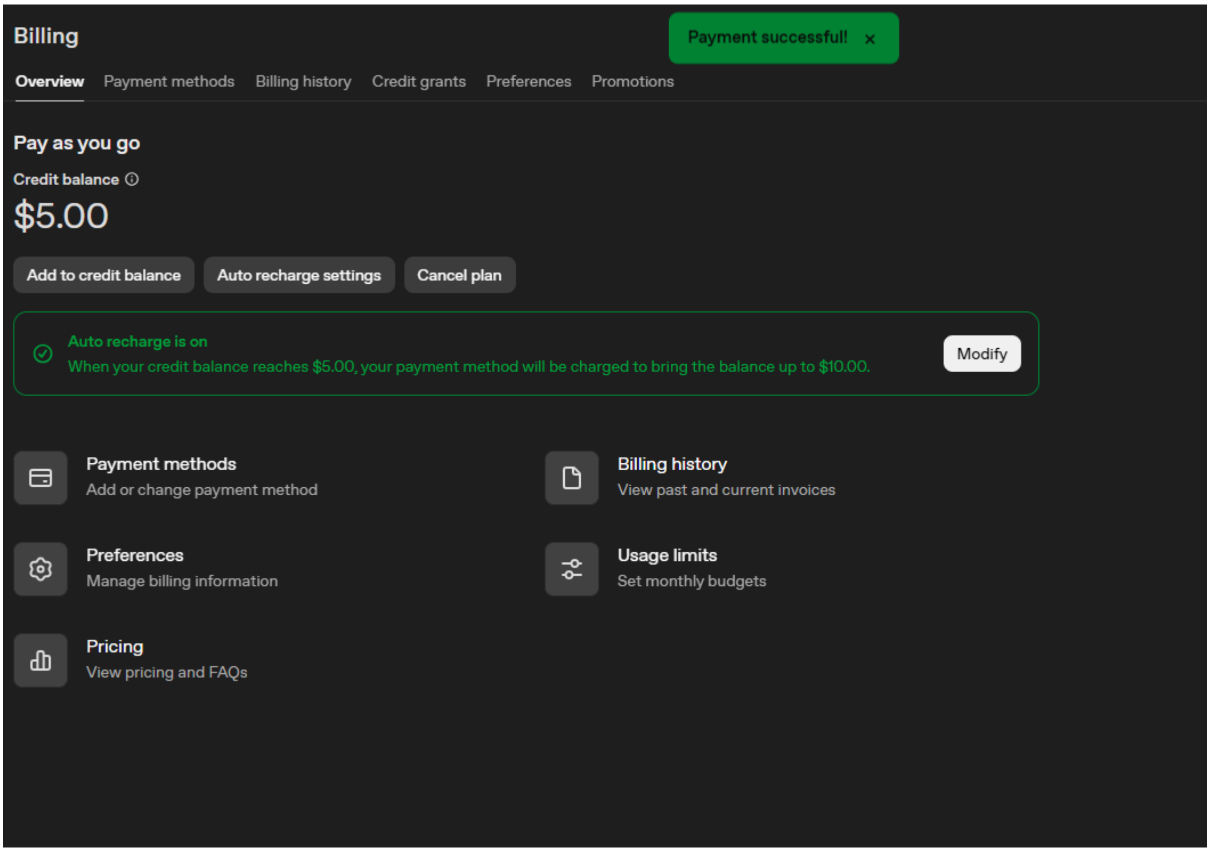
      From a single event (Trigger), the n8n platform now performs two completely different and independent actions at the same time!
- **Document Loader:** The vector base needs raw data. Look at the bottom of the Pinecone node; you will see some lower connection ports.
  - Click on the named port **Document** and choose **Default Data Loader.** This will successfully retrieve the massively downloaded document on the new thread.
- **Text Splitter - Chunking:** Because we are working with large documents (e.g. hundreds of log lines), we need to cut them.
  - Look at the sub-node *Default Data Loader* that you just added. On its bottom port (the gray connection point called *Text Splitter*), click, search **Recursive Character Text Splitter** and select it to physically attach it.
  - Set **Chunk Size** (Fragment size) to 1000 and **Chunk Overlap** (Overlap) to 100.
- **Translator (Embeddings):** Now click on the lower left port of the main Pinecone node, named **Embeddings**. Search and choose **OpenAI Embeddings**.
  - **Step A (Invoicing):** Open a new tab, go to platform.openai.com and log in.*(Note: Your ChatGPT Plus account does not apply here; the API platform is completely separate)*. From the left menu, go to **Settings -> Billing -> Add to balance** and add the minimum credit (e.g. $5). This amount will last you for months of testing. Without it, OpenAI will block vector generation.



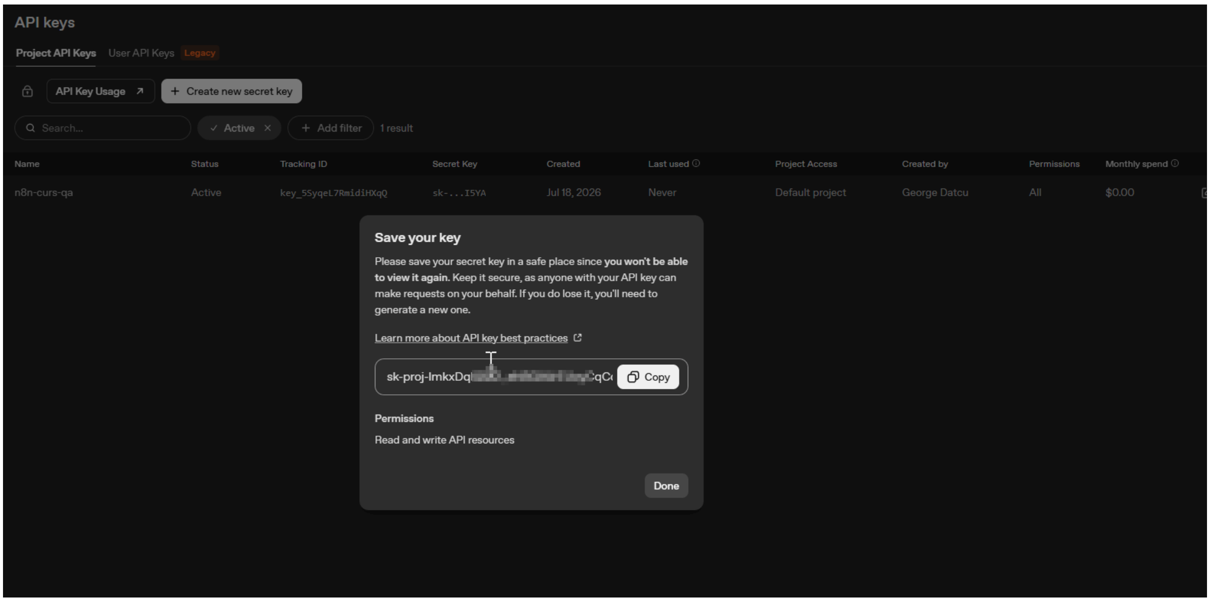
platform.openai.com/settings/organization/billing/overview
Back to project
General
API keys
Admin keys
People
Projects
Billing
Limits
Tunnels
Usage
Service health
Data controls
Security
Add credits
Run your next API request by adding credits.
Go to Billing
Personal
Organization
Billing
Overview
Payment methods
Billing history
Credit grants
Preferences
Promotions
Free trial
Credit remaining
$0.00
Add payment details
View usage
Note: This does not reflect the status of your ChatGPT account.
Payment methods
Add or change payment method
Preferences
Manage billing information
Pricing
View pricing and FAQs
Configure payment
Initial credit purchase
$ 5
Enter an amount between $5 and $100
Model pricing
Auto recharge
Use auto recharge
Automatically add credits when your balance runs low.
When balance drops to:
$ 5
Restore balance to:
$ 10
Recharge amount:
$ 5.00
Monthly recharge limit
Recharging stops once this limit is reached. Your usage tier defines the maximum limit ($100), but you can set a lower limit.
Recharge will happen immediately
Your credit balance is below the recharge threshold you've set, so a recharge will happen immediately after you save these settings.
Back
Continue

QualiAdept


Billing
Overview
Payment methods
Billing history
Credit grants
Preferences
Promotions
Free trial
Credit remaining
$0.00
Add payment details
View usage
Note: This does not reflect the status of your ChatGPT accou
Payment methods
Add or change payment method
Preferences
Manage billing information
Pricing
View pricing and FAQs
Payment summary
Due today
Description
Amount
OpenAI API usage credit
$5.00
Estimated tax
$1.05
Estimated total
$6.05
Payment method
····8799
Expires 11/2029
Default
By continuing you agree to our service credit terms. Paid credits are non-refundable and expire one year from purchase date.
Back
Confirm payment

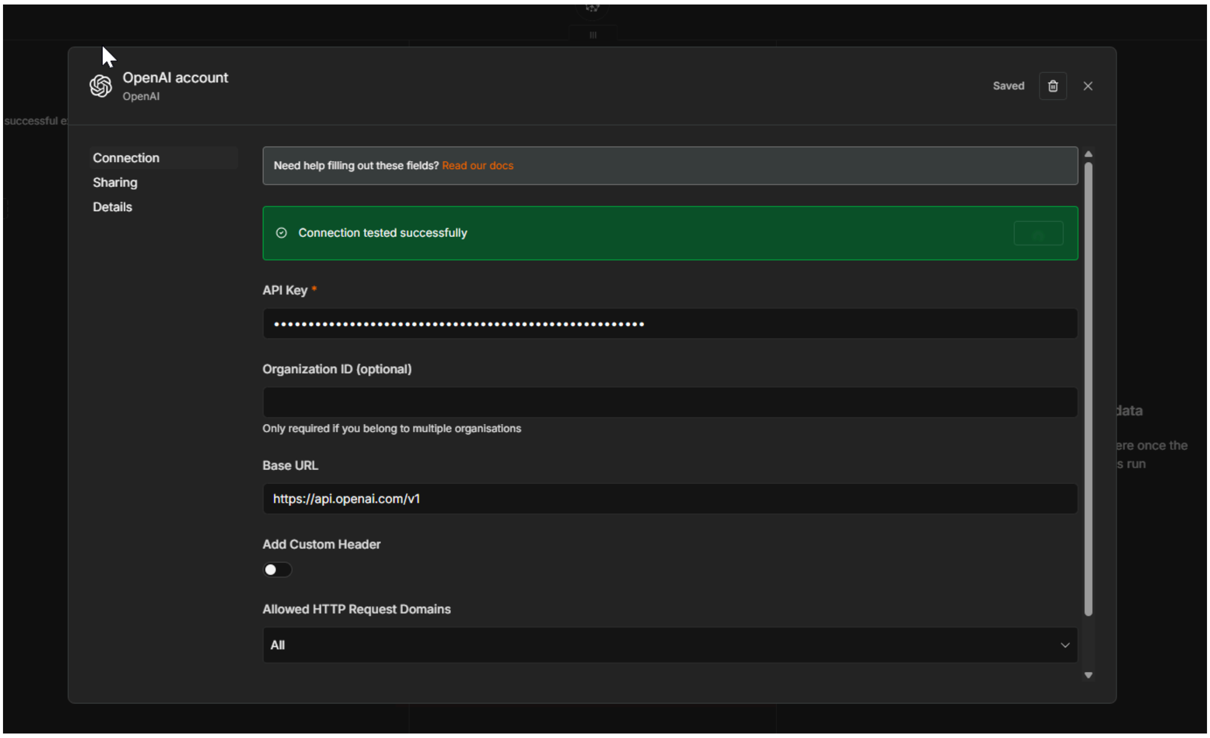


- **Step B (Key):** Go to the main menu at **API Keys**. Press the button **Create new secret key** (you can call it n8n-course).

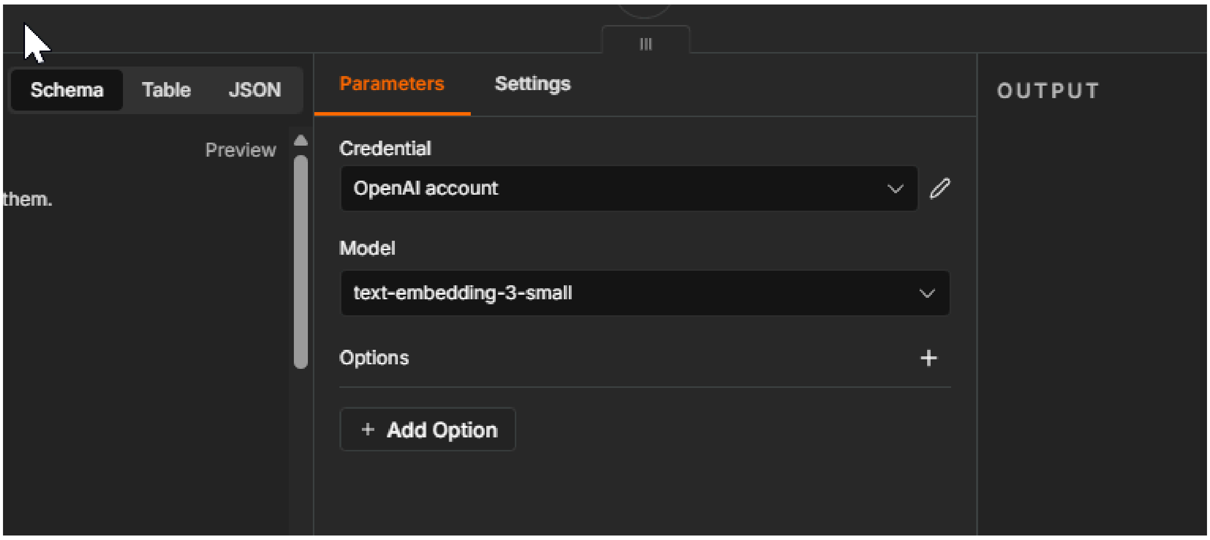


- **Step C (Danger!):** As soon as you press Create, OpenAI will display your long key on the screen once. **Press the Copy button immediately!** If you close the window, you will never be able to see it again and will have to generate another one.
- **Step D (Connection):** Return to n8n at the sub-node *OpenAI Embeddings*. At the

option *Credential*, select *Create New Credential*, paste the key copied in the previous step and press Save.



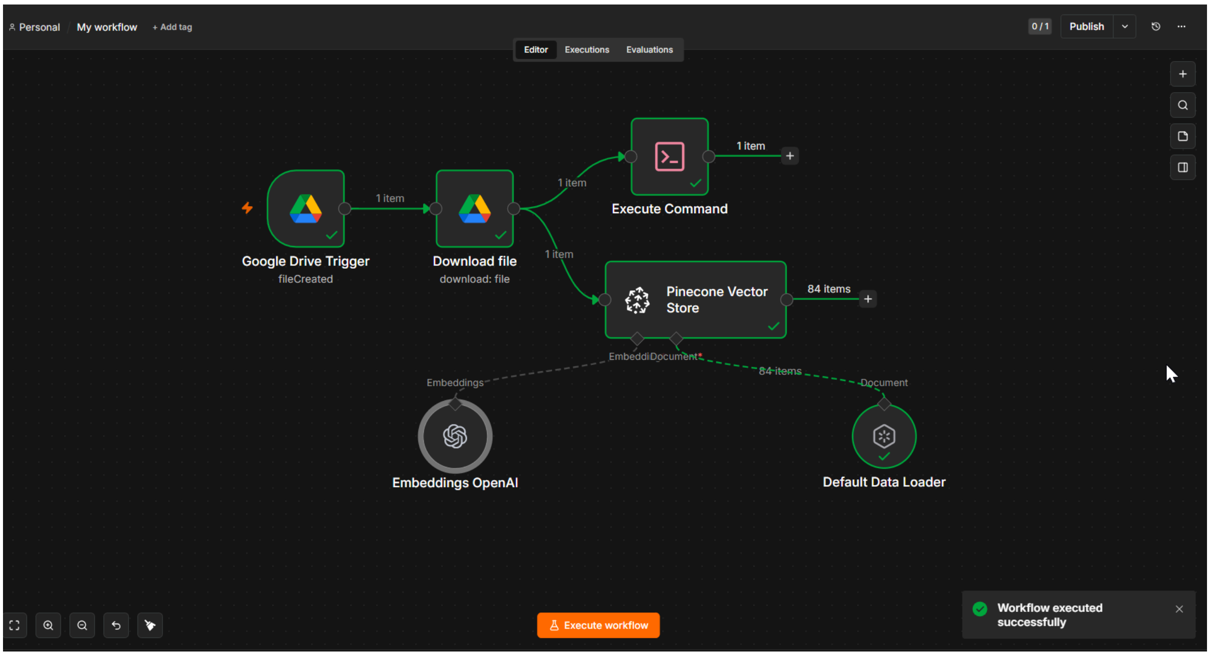
- **Step E:** At the parameter*Model*, select text-embedding-3-small. It will generate the vectors at the exact size of 1536 requested by Pinecone.



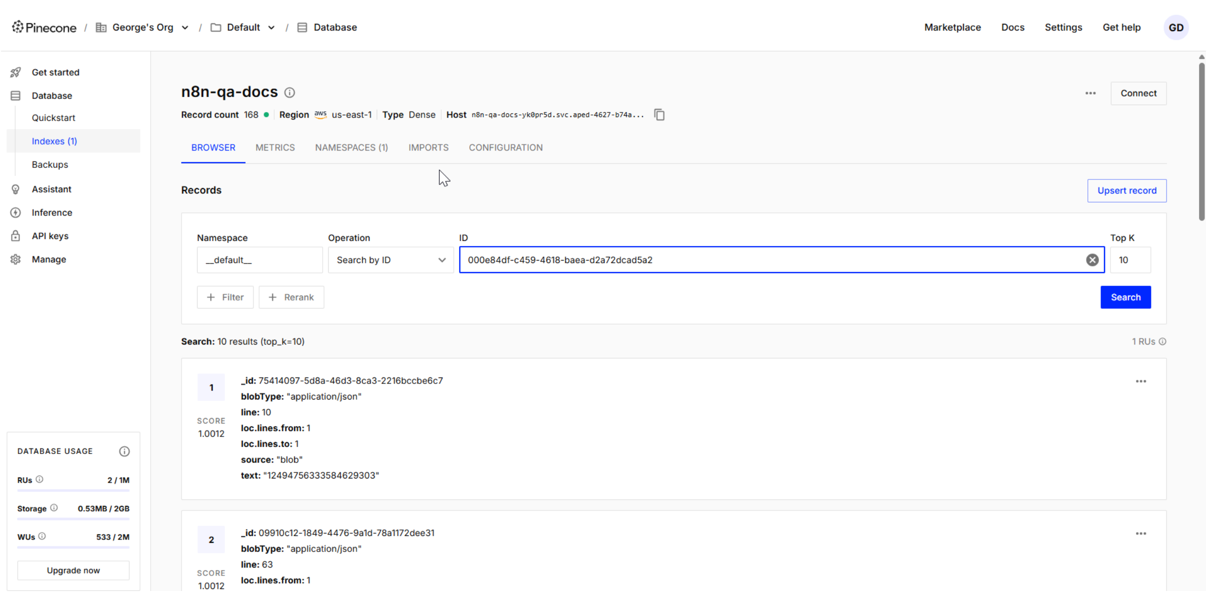
5. The Moment of Truth: Execution and Validation of Ingestion

Our hard work was not in vain! Now that the pipe is connected correctly (on the forked branch), it's time to send your file to AI and Pinecone.

- Make sure you have an actual text file in the folder in Google Drive.
- Go to your main node, **Pinecone Vector Store**, in n8n.
- Press the orange button with confidence. **Execute step**. Wait a few seconds for it to process; the node should turn green ("Succeeded").



- **Visual validation:** If the execution is successful, open the Pinecone platform tab in your browser and click on the name of your index n8n-qa-docs.
- Look at the top left, under your index name, at the metric **Record count**. You will see that the number has increased from 0 to tens or hundreds of fragments! Also, in the center of the screen, in the tab **BROWSER**, you will see the IDs of the vectors actually stored.



Congratulations! You have successfully written your first vector to an AI-based external memory!

🕵️ **Thought Exercise: "Data Detective"**

> You asked yourself in Step 3 why AI architects use a *Chunk Overlap* (Overlap) of 100 characters when it cuts text? Think of a technical document cut off halfway through a key sentence. If there was no overlap between fragment 1 and fragment 2, what would the AI miss?

✅ Solutions and Answers

Solution to the "Data Detective" exercise:

If we "cut" the fixed text to 1000 characters, we risk cutting a word or technical phrase in two. The *Overlap* of 100 characters takes the end of the first fragment and pastes it at the beginning of the second. This way, the cut phrase will be found intact in both pieces, and the meaning (context) is never lost!